

Coding Assignment 3

ECS 122A Algorithm Design and Analysis

Finding a House in ByteLand

Problem Description

Note: This coding assignment includes 80% test cases that admit much simpler solution than the official full-score solution. Therefore, even if you cannot solve the full problem, you may still solve some of the subtasks to get partial credit. See Constraints below for more details.

ByteLand just got redesigned. Now, it is a board consisting of $R \times C$ cells, organized in R rows and C columns. For instance, for a chess board, we have $R = C = 8$. The rows are indexed 1 through R , and the columns are indexed 1 through C . Cell (i, j) refers to the cell at the i -th row and the j -th column.

There is a kid ADA – some friends also call her “Algorithm-Design-Analysis” but, you know, it is a somewhat cumbersome name – who is located at cell $(1, 1)$. Her home is at position (R, C) , and she really really wants to get home as soon as possible for her favorite show – believe it or not, finally, after the ages, the show is going to reveal a new chapter, called “The Winds of Winter”!!! It is instructive to think of cell $(1, 1)$ as the most north-west cell and of (R, C) as the most south-east cell.

From cell (i, j) ADA can move to cells $(i + 1, j)$, $(i - 1, j)$, $(i, j + 1)$ or $(i, j - 1)$. The time she spends depends on the cell she enters, as explained next.

Navigating through the redesigned ByteLand is not that easy. There are several restrictions and obstacles. An obvious restriction is that ADA can never leave ByteLand – okay, to be frank, she hypothetically can leave, but she does not want to because she says so! That is, at any time, she has to be on one of the $R \times C$ cells.

Buuuuut, that is not all. Some cells are easily passable, and entering such a cell takes 1 second. We call these cells empty – as empty as ADA’s hopes that any chapter of “A Dream of Spring” will ever be released.

Some cells have rocks, and at this point, ADA does not wear proper shoes to walk over, let alone climb, rocks. Some cells are muddy. You can think of such cells as being surrounded by mud, so entering them takes longer. For each muddy cell, it takes ADA between 2 and 5 seconds to enter it – interestingly enough, that number of seconds is always an integer.

Once ADA is in a cell, moving around within that cell does not take extra time; only the cell she enters next contributes to the total time. In particular, since ADA starts at $(1, 1)$, we have $\text{result}(1, 1) = 0$. The examples below further illustrate this convention.

If you help ADA find the fastest path, i.e., the path taking the least time, from $(1, 1)$ to (R, C) , she will help you solve your other assignments ;). In particular, she just needs for each cell (i, j) , the shortest amount of time needed to reach (i, j) . She already knows how to use this information to construct all shortest paths and – perhaps more importantly – a random shortest path. All shortest paths are equally important in her eyes, and she does not want to trust anyone (let alone your computer program) to be unbiased!

Input and Output

Input

Each input file contains several test instances. The first line contains a single integer I , representing the number of test instances. The description of the I instances follows.

Each test instance consists of multiple lines:

- The first line contains two space-separated integers R and C . R subsequent lines follow.
- The next R lines contain C space-separated integers denoting the types of the cells in row-major order. The j^{th} integer in the i^{th} row is the type of the cell (i, j) . We denote it by $\text{grid}(i, j)$.
- If $\text{grid}(i, j) = 1$, then cell (i, j) is an empty cell,
- If $\text{grid}(i, j) \in [2, 5]$ then it is a muddy cell such that entering it takes ADA $\text{grid}(i, j)$ seconds,
- If $\text{grid}(i, j) = 0$, then it is a rocky cell,

Output

Compute **result**, an $R \times C$ matrix where $\text{result}(i, j)$ denotes the minimum amount of time ADA needs to reach cell (i, j) from $(1, 1)$. If (i, j) **cannot** be reached from $(1, 1)$, then output **-1** instead. Print the matrix as R lines where each line contains C space separated integers. These integers should be the elements of the **result** matrix in row-major order.

Constraints

Each input file contains at most 11 test instances. In a single input file, the total number of cells across all test instances is at most $4 \cdot 10^6$. The time limit (for all test instances in a single file) is 5 seconds for C/C++ and 10 seconds for Python.

Each test instance satisfies the following additional constraints:

- Cell $(1, 1)$ is empty, i.e., $\text{grid}(1, 1) = 1$
- $1 \leq R, C \leq 1000$
- Sum of $R \cdot C$ across all instances is at most $4 \cdot 10^6$
- In 80% of test files, each cell is either empty or rocky. Moreover, in two of those test files, R and C are at most 10 for each test instance.

Test Cases

Your program will be evaluated on 10 test files. Passing all the instances within a file will earn you 0.8 points. Note that if your program outputs an incorrect answer for even one of the instances within a file, you will receive 0 points for that file.

Sample Input and Output

Note: In the samples below, extra spaces and new lines are added to improve readability. In the actual cases, there are no extra new lines between test cases or extra spaces in the grid output to make sure the cell values line up in a single column.

Sample Input 1:

```
3
2 5
1 0 1 1 1
1 1 1 0 1
```

```

2 5
1 5 1 1 1
1 1 1 5 1

```

```

2 4
1 4 0 1
1 3 1 5

```

Sample Output 1:

```

0 -1 4 5 6
1 2 3 -1 7

```

```

0 5 4 5 6
1 2 3 8 7

```

```

0 4 -1 11
1 4 5 10

```

Sample Explanation

In the following, we provide explanations on the shortest path needed to get to the bottom-rightmost cell, i.e., (R, C) .

Test Case 1: There is a unique path from $(1, 1)$ to every reachable cell in this case. ADA must avoid the rocky cells at $(1, 2)$ and $(2, 4)$, so the reachable cells form a single corridor. Since ADA starts at $(1, 1)$, we have $\text{result}(1, 1) = 0$, and each next cell on that corridor increases the value by 1.

Test Case 2: The cells $(1, 2)$ and $(2, 4)$ are muddy, and entering either of them takes 5 seconds. Even so, the fastest path to $(R, C) = (2, 5)$ is still the same as in Test Case 1, because that path avoids both muddy cells. This gives $\text{result}(2, 5) = 7$. The values $\text{result}(1, 2) = 5$ and $\text{result}(2, 4) = 8$ illustrate the additional time needed to enter muddy cells.

Test Case 3: Cell $(1, 3)$ is rocky, so ADA cannot use the top row all the way to the end. Therefore, to reach the bottom-right cell, she must go through the bottom row. The optimal path is $(1, 1) \rightarrow (2, 1) \rightarrow (2, 2) \rightarrow (2, 3) \rightarrow (2, 4)$, whose total time is $1 + 3 + 1 + 5 = 10$. This example illustrates that the shortest path may still need to go through muddy cells.

Submission Guideline

Write your program in either C, C++, or Python **in a single file**. Submit the file on Gradescope. The time limit on Gradescope is 5 seconds for C/C++ and 10 seconds for Python. You can make at most 10 submission attempts. You may refer to `sample-cpp.cpp` or `sample-py.py` for sample code that takes input and writes output.